

STATIC MALWARE ANALYSIS

2025-06-13 - TRAFFIC ANALYSIS EXERCISE: IT'S A TRAP!

Introduction

From this PCAP file I obtained from <https://www.malware-traffic-analysis.net/index.html>, I hope I can sharpen my knowledge in how malware behaves at a packet-level point of view. And later when I am done with my analysis, I will answer the 4 questions that are included in the webpage of this particular malware.

Below is the information about the packet capture I am analyzing from malware traffic analysis website.

- LAN segment range: **10.6.13[.]0/24** (**10.6.13[.]0** through **10.6.13[.]255**)
- Domain: **massfriction[.]com**
- Active Directory (AD) domain controller: **10.6.13[.]3 - WIN-DQL4WFWJXQ4**
- AD environment name: **MASSFRICTION**
- LAN segment gateway: **10.6.13[.]1**
- LAN segment broadcast address: **10.6.13[.]255**

Methodology

I will be performing the analysis on a Virtual Machine using Wireshark, and see findings I can find and find out what the malware intends to do in the infected machine.

I already downloaded the zip file from the link given below:

<https://www.malware-traffic-analysis.net/2025/06/13/index.html>

If you want to do this, make sure to do it in a Virtual machine or controlled environment, since I will not be responsible for any damage to anyone's laptop due to this project.

Technically, it is possible to install Wireshark on the host machine, but it can be risky as the malware has a possibility to escape out of the packet capture. But it is possible to interact with the packet capture by opening it in Wireshark, but do not extract anything out of the packet capture, and then executing it as it may contain the malware.

Here are some settings I will recommend for the Virtual machine for the best of safety:

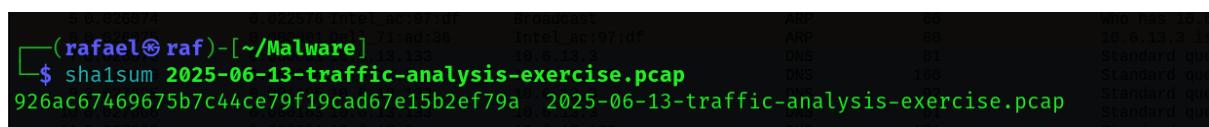
- Internal network instead of bridged adapter to keep it isolated
- Turn off shared clipboard

The screenshot I have used for this project is taken by the Snipping Tool from my host Windows machine.

I will also be using VirusTotal from <https://www.virustotal.com> in order to scan links, whether they are malicious or not. They check it by checking the hash (usually SHA256), and they compare it to the billions of other files, and notice if it is ever marked as malicious.

Results

I have extracted the zip file using the unzip command tool, and there is only a PCAP file inside of it. But, below is the sha256 hash for this packet capture file.



```
(rafael@raf)-[~/Malware]
$ sha1sum 2025-06-13-traffic-analysis-exercise.pcap
926ac67469675b7c44ce79f19cad67e15b2ef79a 2025-06-13-traffic-analysis-exercise.pcap
```

From the TCP conversations in the PCAP, we can see it is only happening between 2 IP addresses, which are

- 10.6.13.13
- 10.6.13.133

It can also be observed that some of the packets were sent at a very high port number which is suspicious, and some of them were port 445 which is SMB.

Conversation Settings	Ethernet - 1	IPv4 - 1	TCP - 55	UDP - 131
Name resolution	Address A	Port A Address B	Port B	Packets
Absolute start time	10.6.13.133	445 10.6.13.133	52423	2 120 bytes
Limit to display filter	10.6.13.133	52427 10.6.13.133	389	15 618
Copy	10.6.13.133	52428 10.6.13.133	88	12 418
Follow Stream...	10.6.13.133	52433 10.6.13.133	88	8 899 bytes
Graph...	10.6.13.133	52434 10.6.13.133	88	10 318
Protocol	10.6.13.133	52435 10.6.13.133	88	12 418
Bluetooth	10.6.13.133	52436 10.6.13.133	445	64 1718
BPV7	10.6.13.133	52437 10.6.13.133	135	14 218
DCCP	10.6.13.133	52438 10.6.13.133	49669	22 618
Ethernet	10.6.13.133	52439 10.6.13.133	88	12 418
FC	10.6.13.133	52440 10.6.13.133	88	12 418
FDDI	10.6.13.133	52441 10.6.13.133	88	12 418
IEEE 802.11	10.6.13.133	52442 10.6.13.133	59774	12 318
IEEE 802.15.4	10.6.13.133	52443 10.6.13.133	389	19 818
IPv4	10.6.13.133	52444 10.6.13.133	389	15 418
IPv6	10.6.13.133	52445 10.6.13.133	445	166 2418
IPX	10.6.13.133	52446 10.6.13.133	88	12 418
JXTA	10.6.13.133	52454 10.6.13.133	135	11 218
LTP	10.6.13.133	52455 10.6.13.133	49669	23 718
MPTCP	10.6.13.133	52456 10.6.13.133	389	21 818
Filter list for specific type	10.6.13.133	52457 10.6.13.133	389	16 718
	10.6.13.133	52459 10.6.13.133	445	24 818
	10.6.13.133	52504 10.6.13.133	135	11 218
	10.6.13.133	52505 10.6.13.133	49669	23 718
	10.6.13.133	52506 10.6.13.133	389	21 818
	10.6.13.133	52507 10.6.13.133	389	16 718
	10.6.13.133	52535 10.6.13.133	135	12 118
	10.6.13.133	52536 10.6.13.133	49669	20 518
	10.6.13.133	52537 10.6.13.133	389	15 718
	10.6.13.133	52538 10.6.13.133	88	12 418
	10.6.13.133	52539 10.6.13.133	389	11 318

As we can see from the Wireshark screenshot below on packet number 8-12 it is making a DNS query to massfriction.com. With Web-Proxy Auto Discovery which is a technology that allows browsers to detect proxy servers, in this case it is finding for the domain of massfriction.com. And we see a Windows host name which is being resolved, whose name is DESKTOP-5AVE44C who belongs to 10.6.13.133.

No.	Time	Delta	Source	Destination	Signal	Protocol	Length	TCP Segment Len	Info
7	0.026076	0.000000	10.6.13.133	10.6.13.133	DNS	81			Standard query 0x2b8f A wpad.massfriction.com
8	0.026430	0.000354	10.6.13.133	10.6.13.133	DNS	160			Standard query response 0x2b8f No such name A wpad.massfriction.com SOA win-d
9	0.027625	0.001195	10.6.13.133	10.6.13.133	DNS	92			Standard query 0x56e9 SOA DESKTOP-5AVE44C.massfriction.com
10	0.027808	0.000183	10.6.13.133	10.6.13.133	DNS	81			Standard query 0x4900 A wpad.massfriction.com
11	0.027899	0.000091	10.6.13.133	10.6.13.133	DNS	171			Standard query response 0x56e9 SOA DESKTOP-5AVE44C.massfriction.com SOA win-d
13	0.028019	0.000210	10.6.13.133	10.6.13.133	DNS	160			Standard query response 0x4900 No such name A wpad.massfriction.com SOA win-d
14	0.028999	0.000971	10.6.13.133	10.6.13.133	DNS	160			Dynamic update 0x544f SOA massfriction.com CNAME AAAA A 10.6.13.133
15	0.031007	0.002017	10.6.13.133	10.6.13.133	DNS	160			Dynamic update response 0x544f SOA massfriction.com CNAME AAAA A 10.6.13.133
16	0.047720	0.016713	10.6.13.133	10.6.13.133	DNS	128			Standard query 0x6fad SRV _ldap._tcp.Default-First-Site-Name._sites._msdc
17	0.048016	0.000296	10.6.13.133	10.6.13.133	DNS	196			Standard query response 0x6fad SRV _ldap._tcp.Default-First-Site-Name._sites._
18	0.048440	0.000424	10.6.13.133	10.6.13.133	DNS	92			Standard query 0x8507 A win-dql4wfwjxq4.massfriction.com
19	0.049063	0.000622	10.6.13.133	10.6.13.133	DNS	108			Standard query response 0x8507 A win-dql4wfwjxq4.massfriction.com A 10.6.13.133
20	0.049353	0.000300	10.6.13.133	10.6.13.133	CLDAP	269			searchRequest(82) "croot" baseObject
21	0.049355	0.000002	10.6.13.133	10.6.13.133	CLDAP	269			searchRequest(83) "croot" baseObject
22	0.049604	0.000249	10.6.13.133	10.6.13.133	CLDAP	244			searchResEntry(82) "croot" searchResDone(82) success [2 results]
23	0.049605	0.000001	10.6.13.133	10.6.13.133	CLDAP	244			searchResEntry(83) "croot" searchResDone(83) success [2 results]
28	0.159966	0.117361	10.6.13.133	10.6.13.133	TCP	66			0 52427 -> 389 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
29	0.159969	0.000003	10.6.13.133	10.6.13.133	TCP	66			0 389 -> 52427 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
30	0.157393	0.000334	10.6.13.133	10.6.13.133	TCP	60			0 52427 -> 389 [ACK] Seq=1 Ack=1 Win=65280 Len=0
31	0.157547	0.000244	10.6.13.133	10.6.13.133	LDAP	404			359 searchRequest(9) "croot" baseObject
32	0.158447	0.000900	10.6.13.133	10.6.13.133	TCP	1514			1460 389 -> 52427 [ACK] Seq=1 Ack=351 Win=1049344 Len=1460 [TCP PDU reassembled in
33	0.158449	0.000002	10.6.13.133	10.6.13.133	LDAP	1430			1376 searchResEntry(9) "croot" searchResDone(9) success [1 result]
34	0.158619	0.000170	10.6.13.133	10.6.13.133	TCP	60			0 52427 -> 389 [ACK] Seq=351 Ack=2837 Win=65280 Len=0
35	0.159044	0.000425	10.6.13.133	10.6.13.133	TCP	66			0 52427 -> 389 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
36	0.159946	0.000902	10.6.13.133	10.6.13.133	TCP	66			0 88 -> 52428 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM

Here is the protocol hierarchy of the packet capture file. And we can see most of the conversations are made under the TCP protocol, approximately around 96%. And specifically, most of them were the protocol of Transport Layer Security (TLS).

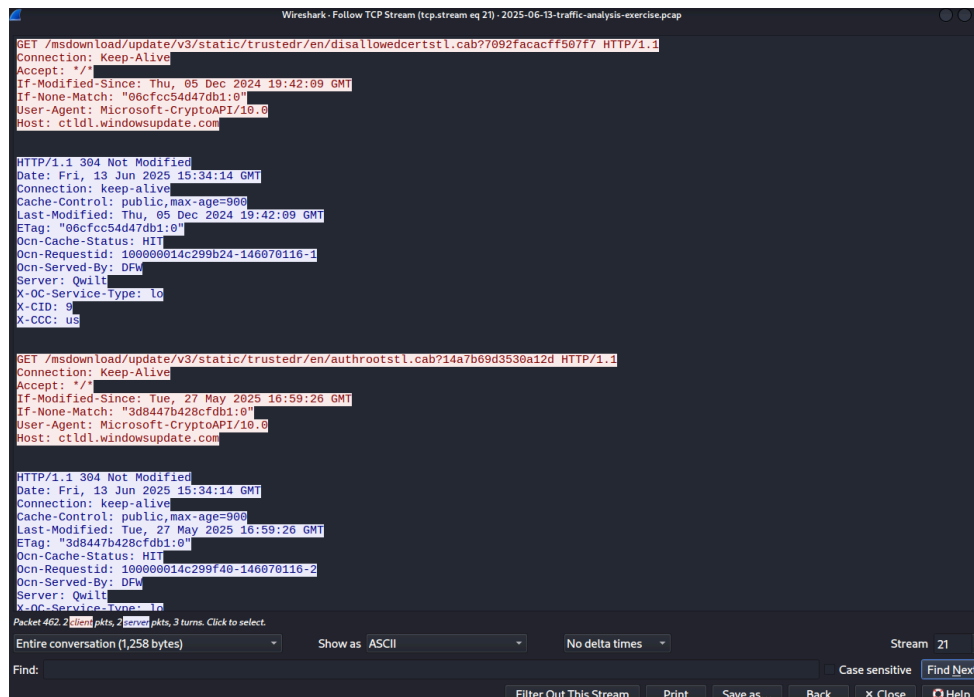
Rafael Angelo Christianto

Wireshark - Protocol Hierarchy Statistics - 2025-06-13-traffic-analysis-exercise.pcap									
Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDU's
Frame	100.0	48877	100.0	42106050	162 k	0	0	0	48877
Ethernet	100.0	48877	1.9	783768	3,025	0	0	0	48877
Link Layer Discovery Protocol	0.0	2	0.0	106	0	2	106	0	2
Internet Protocol Version 4	99.5	48615	2.3	972300	3,753	0	0	0	48615
User Datagram Protocol	2.7	1302	0.0	10416	40	0	0	0	1302
Simple Service Discovery Protocol	0.0	15	0.0	1935	7	15	1935	7	15
QUIC IETF	1.5	748	1.2	494338	1,908	748	488412	1,885	754
Network Time Protocol	0.0	6	0.0	720	2	6	720	2	6
NetBIOS Name Service	0.1	33	0.0	2172	8	33	2172	8	33
NetBIOS Datagram Service	0.1	72	0.0	5904	22	0	0	0	72
SMB (Server Message Block Protocol)	0.1	72	0.0	7818	30	0	0	0	72
SMB MailSlot Protocol	0.1	72	0.0	1800	6	0	0	0	72
Microsoft Windows Browser Protocol	0.1	72	0.0	1626	6	72	1626	6	72
Multicast Domain Name System	0.0	3	0.0	120	0	3	120	0	3
Link-local Multicast Name Resolution	0.0	2	0.0	66	0	2	66	0	2
eXtensible Markup Language	0.0	11	0.0	9268	35	11	9268	35	11
Dynamic Host Configuration Protocol	0.0	4	0.0	1247	4	4	1247	4	4
Domain Name System	0.8	392	0.1	34758	134	392	34758	134	392
Connectionless Lightweight Directory Access Protocol	0.0	16	0.0	3475	13	16	3475	13	16
Transmission Control Protocol	96.8	47300	2.3	981508	3,788	33380	703108	2,714	47300
Transport Layer Security	26.6	12979	83.7	35226952	135 k	12974	25955436	100 k	13327
NetBIOS Session Service	0.3	165	0.1	47747	184	0	0	0	165
SMB2 (Server Message Block Protocol version 2)	0.3	161	0.1	46467	179	129	37941	146	171
SMB (Server Message Block Protocol)	0.0	4	0.0	620	2	4	620	2	4
Lightweight Directory Access Protocol	0.3	171	0.3	122871	474	171	91917	354	182
Kerberos	0.0	18	0.1	26795	103	18	26795	103	18
Hypertext Transfer Protocol	0.2	76	0.0	17059	65	8	2195	8	76
Line-based text data	0.0	3	2.4	996810	3,847	3	996810	3,847	3
eXtensible Markup Language	0.0	2	0.0	2941	11	2	2941	11	2
Distributed Computing Environment / Remote Procedure Call (DCE/RPC)	0.5	222	0.2	63743	246	58	28241	109	222
SAMR (pidl)	0.1	30	0.0	2226	8	30	2226	8	30
Microsoft Network Logon	0.0	2	0.0	1888	7	2	1888	7	2
DCE/RPC Endpoint Mapper	0.1	34	0.0	6692	25	34	6692	25	34
Active Directory Replication	0.2	98	0.0	12400	47	98	12400	47	98

Packet Capture Analysis (clean) [Running] - Oracle VirtualBox									
File Machine View Input Devices Help									
2025-06-13-traffic-analysis-exercise.pcap									
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help									
http									
No.	Time	Source	Destination	Signal	Protocol	Length	Info	Stream ID	
126	4.723359	10.6.13.133	23.192.223.206		HTTP	165	GET /connecttest.txt HTTP/1.1		
133	4.825109	23.192.223.206	10.6.13.133		HTTP	241	HTTP/1.1 200 OK (text/plain)		
462	18.932659	10.6.13.133	217.20.51.22		HTTP	341	GET /msdownload/update/v3/static/trustedr/en/disallowedcertstl.cab?7092facacff507f7 HTTP/1.1		
464	18.965580	217.20.51.22	10.6.13.133		HTTP	398	HTTP/1.1 304 Not Modified		
466	19.081208	10.6.13.133	217.20.51.22		HTTP	336	GET /msdownload/update/v3/static/trustedr/en/authrootstl.cab?14a7b69d3530a12d HTTP/1.1		
467	19.115178	217.20.51.22	10.6.13.133		HTTP	399	HTTP/1.1 304 Not Modified		
5979	86.252259	10.6.13.133	10.6.13.129		HTTP	787	POST /e762ae39-f18a-00e7-ba88-e8ff9d15458c/ HTTP/1.1		
5982	86.255408	10.6.13.129	10.6.13.133		HTTP	945	HTTP/1.1 200		
6642	113.131635	10.6.13.133	104.21.24.186		HTTP	233	GET /zh66PZdkt HTTP/1.1		
6654	115.323393	104.21.24.186	10.6.13.133		HTTP	1087	HTTP/1.1 200 OK (text/plain)		
6699	122.774461	10.6.13.133	104.21.112.1		HTTP	92	POST /NV4RqNEU HTTP/1.1		
8561	149.326987	104.21.112.1	10.6.13.133		HTTP	368	HTTP/1.1 200 OK (text/plain)		
442	209.713931	10.6.13.133	104.21.16.1		HTTP	368	POST /YSEpJ/CPZldd829d9c190fd4799508711a44a9c358fa/R5n6W HTTP/1.1		
444	239.812819	10.6.13.133	104.21.16.1		HTTP	418	POST /L1092K6Fa/7qunUIPqCFL92d1798d9439a1582f369aaae9e6f5/MPF2umVY2Thew2E HTTP/1.1		
444	269.891371	10.6.13.133	104.21.16.1		HTTP	399	POST /s3wNeqH/0e1521b1b6cdd449a1d36b097f9cdf2430/dmHL HTTP/1.1		
445	300.027565	10.6.13.133	104.16.230.132		HTTP	421	POST /bmVLT6uabCC/22JpMh12LgaCY48354F9348F3b439a1433a5327275539f4r HTTP/1.1		
446	300.231317	10.6.13.133	104.21.16.1		HTTP	396	POST /VAV056uLBkt2Tndd120b0d89e343591142f2c3d99671d HTTP/1.1		
447	360.282926	10.6.13.133	104.21.16.1		HTTP	371	POST /zj6/95rep831205efb7d9469b1852377d9e86394ad HTTP/1.1		
448	390.364199	10.6.13.133	104.16.230.132		HTTP	446	POST /GTlBbFAxRm28/Wl08353db0b0893b44d954fd148b3f5662db/G04whWesne HTTP/1.1		
448	420.467379	10.6.13.133	104.21.16.1		HTTP	388	POST /xhuuU98DQ/PsgL8312492084d469a1c5acddcf4a47d4f19cu HTTP/1.1		
449	450.568340	10.6.13.133	104.16.230.132		HTTP	437	POST /LChwCfPMNhuU/8yX831c7b1589f344de14d7c6ad3e3d19c3c8/of3qoo3128 HTTP/1.1		
451	480.979688	10.6.13.133	104.21.16.1		HTTP	396	POST /rpyGyah/8153f1b1492dd469b1d3406f2116ca55a9/21 HTTP/1.1		
452	511.068999	10.6.13.133	104.21.16.1		HTTP	399	POST /bck3He5bkn02/R1tm8352091481234b995a626d5b736b72c3b/FwtKu HTTP/1.1		
452	541.338491	10.6.13.133	104.21.16.1		HTTP	415	POST /9wFzVP3bC/K2XonGvFLv1k353c3b1589b0d4797de2089a07b0dc06/a07zf6161X HTTP/1.1		
453	571.426678	10.6.13.133	104.21.16.1		HTTP	384	POST /xpd11tkW/cyE23hDwQVPVUk35298ec92fd47db563aefef2f5ceda2b/1eIM0jfvTYX3 HTTP/1.1		
454	601.547817	10.6.13.133	104.16.230.132		HTTP	418	POST /BqMun/WTMSLH4R34c49348ff344d91caf59e07011ba39c HTTP/1.1		
Frame 126: 165 bytes on wire (1320 bits), 165 bytes captured (1320 bits) on interface 0									
Ethernet II, Src: IntelAc:97:df (24:77:83:ac:97:df), Dst: Cisco54:95:22 (00:02:ba:54:95:22)									
Internet Protocol Version 4, Src: 10.6.13.133, Dst: 23.192.223.206									
Transmission Control Protocol, Src Port: 52431, Dst Port: 80, Seq: 1, Ack: 1, Len: 111									
Hypertext Transfer Protocol									
GET /connecttest.txt HTTP/1.1									
Host: www.msftconnecttest.com									
User-Agent: Microsoft-WebDAV-MiniClient/6.0.6002.1814									
Accept: */*									
Accept-Encoding: gzip, deflate									
Connection: Keep-Alive									
Cache-Control: no-cache									
Content-Type: text/plain									
Content-Length: 111									
Server: Microsoft-IIS/7.5									
X-AspNetMvc-Version: 3.0									
X-AspNet-Version: 4.0.30319									
X-Powered-By: ASP.NET									
Microsoft-WebDAV-MiniClient/6.0.6002.1814									
GET /connecttest.txt HTTP/1.1									
Host: www.msftconnecttest.com									
User-Agent: Microsoft-WebDAV-MiniClient/6.0.6002.1814									
Accept: */*									
Accept-Encoding: gzip, deflate									
Connection: Keep-Alive									
Cache-Control: no-cache									
Content-Type: text/plain									
Content-Length: 111									
Server: Microsoft-IIS/7.5									
X-AspNetMvc-Version: 3.0									
X-AspNet-Version: 4.0.30319									
X-Powered-By: ASP.NET									
Microsoft-WebDAV-MiniClient/6.0.6002.1814									

After we follow the TCP stream of packet number 462, we can see an odd user agent which is Microsoft Crypto API, which is a web crawler by Microsoft, similar to wget or curl.

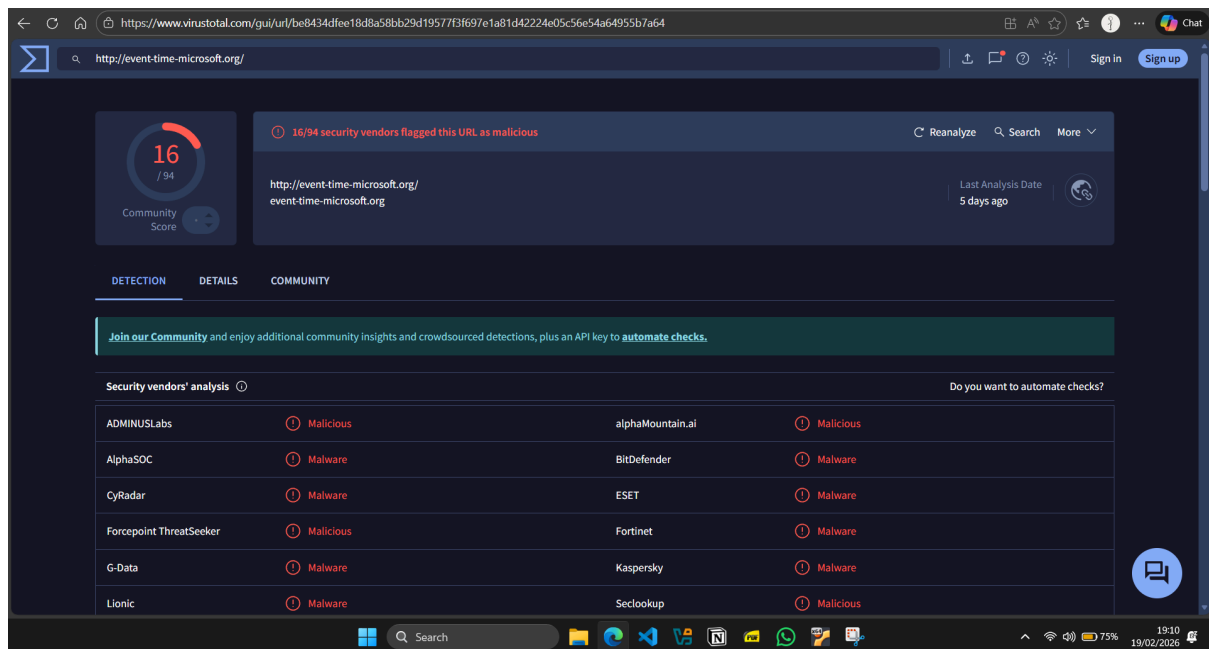
Rafael Angelo Christianto



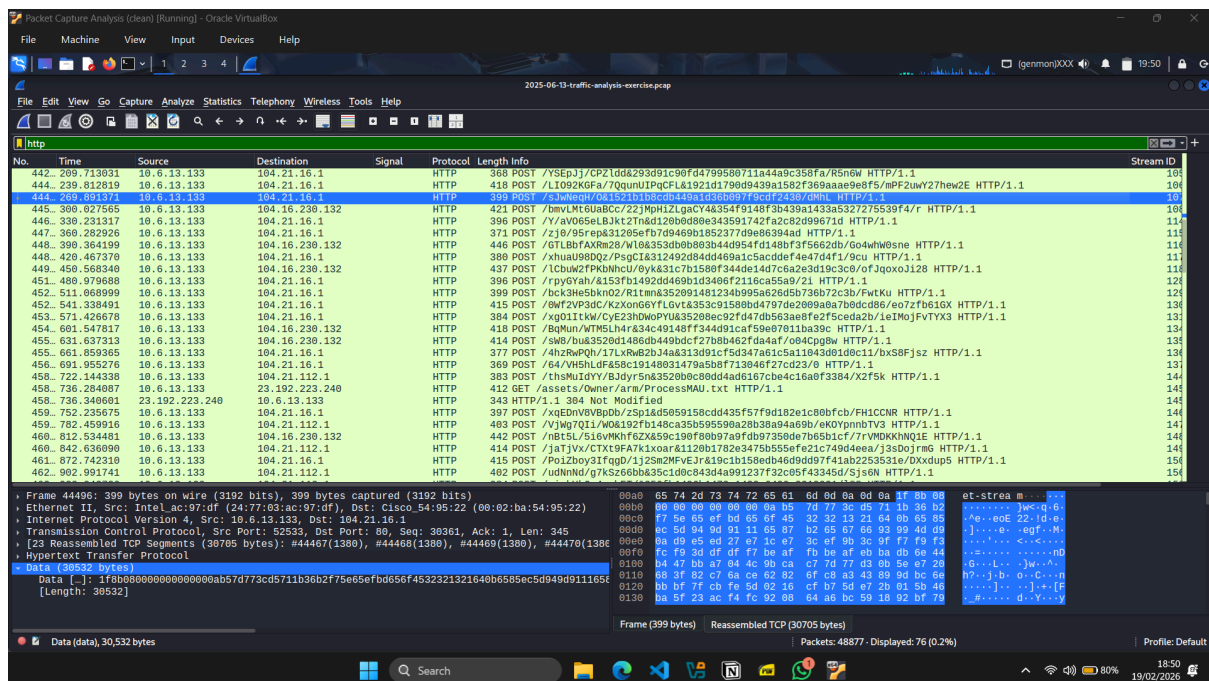
For the image below, this time the user agent is normal but then, the host trying to be reached out is suspicious, which is the: `event-time-microsoft.org`



After checking it at VirusTotal, I found it to be malicious.



From this image below, we can see a lot of the HTTP requests are POST



Now in the screenshot below, packet number 126 , It says a GET connection to a suspicious named file, which is /connecttest.txt. From the file name it is like testing whether we have an internet connection.

No.	Time	Source	Destination	Signal	Protocol	Length	Info
123	2025-06-13 23:34:00.4427380	10.6.13.133	23.192.223.206	TCP	66	52431 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM	
124	2025-06-13 23:34:00.4796955	23.192.223.206	10.6.13.133	TCP	66	80 → 52431 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1396 SACK_PERM WS=128	
125	2025-06-13 23:34:00.4797603	10.6.13.133	23.192.223.206	TCP	60	52431 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0	
126	2025-06-13 23:34:00.4799801	10.6.13.133	23.192.223.206	HTTP	165	GET /connecttest.txt HTTP/1.1	
133	2025-06-13 23:34:00.5117399	23.192.223.206	10.6.13.133	TCP	60	80 → 52431 [ACK] Seq=1 Ack=112 Win=64256 Len=0	
134	2025-06-13 23:34:00.5117400	23.192.223.206	10.6.13.133	HTTP	241	HTTP/1.1 200 OK (text/plain)	
135	2025-06-13 23:34:00.5117420	10.6.13.133	23.192.223.206	TCP	60	80 → 52431 [FIN, ACK] Seq=189 Ack=112 Win=64256 Len=0	
136	2025-06-13 23:34:00.5117422	10.6.13.133	23.192.223.206	TCP	60	52431 → 80 [ACK] Seq=112 Ack=189 Win=65280 Len=0	
137	2025-06-13 23:34:00.5117423	23.192.223.206	10.6.13.133	TCP	60	52431 → 80 [FIN, ACK] Seq=112 Ack=189 Win=65280 Len=0	
137	2025-06-13 23:34:00.5415411	23.192.223.206	10.6.13.133	TCP	60	80 → 52431 [ACK] Seq=189 Ack=113 Win=64256 Len=0	

So we can see the destination IP of that packet 126, which is 23.192.223.206, is for Akamai technologies which is a CDN. So I want to filter again the packets who have the IP 23.192.223.206. The image screenshot will be shown below.

No.	Time	Source	Destination	Signal	Protocol	Length	Info
123	2025-06-13 23:34:00.4427380	10.6.13.133	23.192.223.206	TCP	66	52431 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM	
124	2025-06-13 23:34:00.4796955	23.192.223.206	10.6.13.133	TCP	66	80 → 52431 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1396 SACK_PERM WS=128	
125	2025-06-13 23:34:00.4797603	10.6.13.133	23.192.223.206	TCP	60	52431 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0	
126	2025-06-13 23:34:00.4799801	10.6.13.133	23.192.223.206	HTTP	165	GET /connecttest.txt HTTP/1.1	
133	2025-06-13 23:34:00.5117399	23.192.223.206	10.6.13.133	TCP	60	80 → 52431 [ACK] Seq=1 Ack=112 Win=64256 Len=0	
134	2025-06-13 23:34:00.5117400	23.192.223.206	10.6.13.133	HTTP	241	HTTP/1.1 200 OK (text/plain)	
135	2025-06-13 23:34:00.5117420	10.6.13.133	23.192.223.206	TCP	60	80 → 52431 [FIN, ACK] Seq=189 Ack=112 Win=64256 Len=0	
136	2025-06-13 23:34:00.5117422	10.6.13.133	23.192.223.206	TCP	60	52431 → 80 [ACK] Seq=112 Ack=189 Win=65280 Len=0	
137	2025-06-13 23:34:00.5117423	23.192.223.206	10.6.13.133	TCP	60	52431 → 80 [FIN, ACK] Seq=112 Ack=189 Win=65280 Len=0	
137	2025-06-13 23:34:00.5415411	23.192.223.206	10.6.13.133	TCP	60	80 → 52431 [ACK] Seq=189 Ack=113 Win=64256 Len=0	

So the first 3 packets (123 - 125) are the opening 3 way handshake packets, and then it starts off from 126 where it gets 10.6.13.133 (infected device) gets the connecttest.txt file and it was successful HTTP GET request seen from 133 success code of 200 which translates to successful. Then the last 4 packets (134 - 137) are the closing packets of the connection between the 2 hosts.

I then checked for possible beaconing or C2 behavior by this malware. I went first by filtering for the malicious IP address, which is the 10.6.13.3 IP address. Then after, I went to Statistics > Conversations. The display will be showed like the image below:

Wireshark - Conversations - 2025-06-13-traffic-analysis-exercise.pcap

Conversation Settings	Ethernet - 2	IPv4 - 2	TCP - 55	UDP - 132
Name resolution	Address A	Port A Address B	Port B	Packets Bytes Stream ID Total Packets Percent Filtered Packets A + B Bytes A + B Packets B + A Bytes B + A Rel Start Duration Bits/s A + B
10.6.13.3	445	10.6.13.3	52423	2 120 bytes 70 2 100.00% 1 60 bytes 65.122150 0.0000
10.6.13.3	52427	10.6.13.3	389	15 8 kb 0 15 100.00% 8 3 kb 7 0.159656 0.0107
10.6.13.3	52428	10.6.13.3	88	12 4 kb 1 12 100.00% 6 2 kb 6 0.159844 0.0042
10.6.13.3	52433	10.6.13.3	88	8 899 bytes 6 8 100.00% 4 464 bytes 4 435 bytes 14.222837 0.0014
10.6.13.3	52434	10.6.13.3	88	10 3 kb 7 10 100.00% 5 604 bytes 5 2 kb 14.232483 0.0023
10.6.13.3	52435	10.6.13.3	88	12 4 kb 8 12 100.00% 6 2 kb 6 14.235093 0.0035
10.6.13.3	52436	10.6.13.3	445	64 17 kb 9 64 100.00% 34 10 kb 30 7 kb 14.434042 10.7389
10.6.13.3	52437	10.6.13.3	135	14 2 kb 10 14 100.00% 8 964 bytes 6 1 kb 14.435377 14.2765
10.6.13.3	52438	10.6.13.3	49669	22 6 kb 11 22 100.00% 12 4 kb 10 2 kb 14.437181 34.2755
10.6.13.3	52439	10.6.13.3	88	12 4 kb 12 12 100.00% 6 2 kb 6 2 kb 14.439476 0.0042
10.6.13.3	52440	10.6.13.3	88	12 4 kb 13 12 100.00% 6 2 kb 6 2 kb 14.440878 0.0035
10.6.13.3	52441	10.6.13.3	88	12 4 kb 14 12 100.00% 6 2 kb 6 2 kb 14.443779 0.0021
10.6.13.3	52442	10.6.13.3	59774	12 3 kb 15 12 100.00% 7 2 kb 5 1 kb 14.458735 14.2531
10.6.13.3	52443	10.6.13.3	389	19 8 kb 16 19 100.00% 10 4 kb 9 4 kb 14.480942 0.0563
10.6.13.3	52444	10.6.13.3	389	15 4 kb 17 15 100.00% 8 3 kb 7 993 bytes 14.508608 0.0249
10.6.13.3	52445	10.6.13.3	445	166 24 kb 18 166 100.00% 88 18 kb 78 2041 seconds 2041.0560 85 kbps
10.6.13.3	52446	10.6.13.3	88	12 4 kb 19 12 100.00% 6 2 kb 6 2 kb 14.723946 0.0035
10.6.13.3	52454	10.6.13.3	135	11 2 kb 27 11 100.00% 7 904 bytes 4 872 bytes 28.820104 0.4039
10.6.13.3	52455	10.6.13.3	49669	23 7 kb 28 23 100.00% 13 4 kb 10 2 kb 28.823466 0.4005
10.6.13.3	52456	10.6.13.3	389	21 8 kb 29 21 100.00% 11 4 kb 10 4 kb 28.834403 0.0151
10.6.13.3	52457	10.6.13.3	389	16 7 kb 30 16 100.00% 8 3 kb 8 4 kb 28.844189 0.0045
10.6.13.3	52459	10.6.13.3	445	24 8 kb 32 24 100.00% 14 6 kb 10 2 kb 29.042805 10.7470
10.6.13.3	52504	10.6.13.3	135	11 2 kb 78 11 100.00% 7 904 bytes 4 872 bytes 75.612341 0.6818
10.6.13.3	52505	10.6.13.3	49669	23 7 kb 79 23 100.00% 5 5 kb 10 2 kb 75.620540 0.0736
10.6.13.3	52506	10.6.13.3	389	21 8 kb 80 21 100.00% 11 4 kb 10 4 kb 75.642073 0.0210
10.6.13.3	52507	10.6.13.3	389	16 7 kb 81 16 100.00% 8 3 kb 8 4 kb 75.654903 0.0077
10.6.13.3	52535	10.6.13.3	135	12 1 kb 109 12 100.00% 7 742 bytes 5 670 bytes 315.563454 14.8231
10.6.13.3	52536	10.6.13.3	49669	20 5 kb 110 20 100.00% 11 4 kb 9 2 kb 315.565149 34.8233
10.6.13.3	52537	10.6.13.3	389	15 7 kb 111 15 100.00% 8 3 kb 7 4 kb 317.506895 0.0135
10.6.13.3	52538	10.6.13.3	88	12 4 kb 112 12 100.00% 6 2 kb 6 2 kb 317.510256 0.0038
10.6.13.3	52539	10.6.13.3	389	11 3 kb 113 11 100.00% 6 3 kb 5 715 bytes 317.516771 0.0048

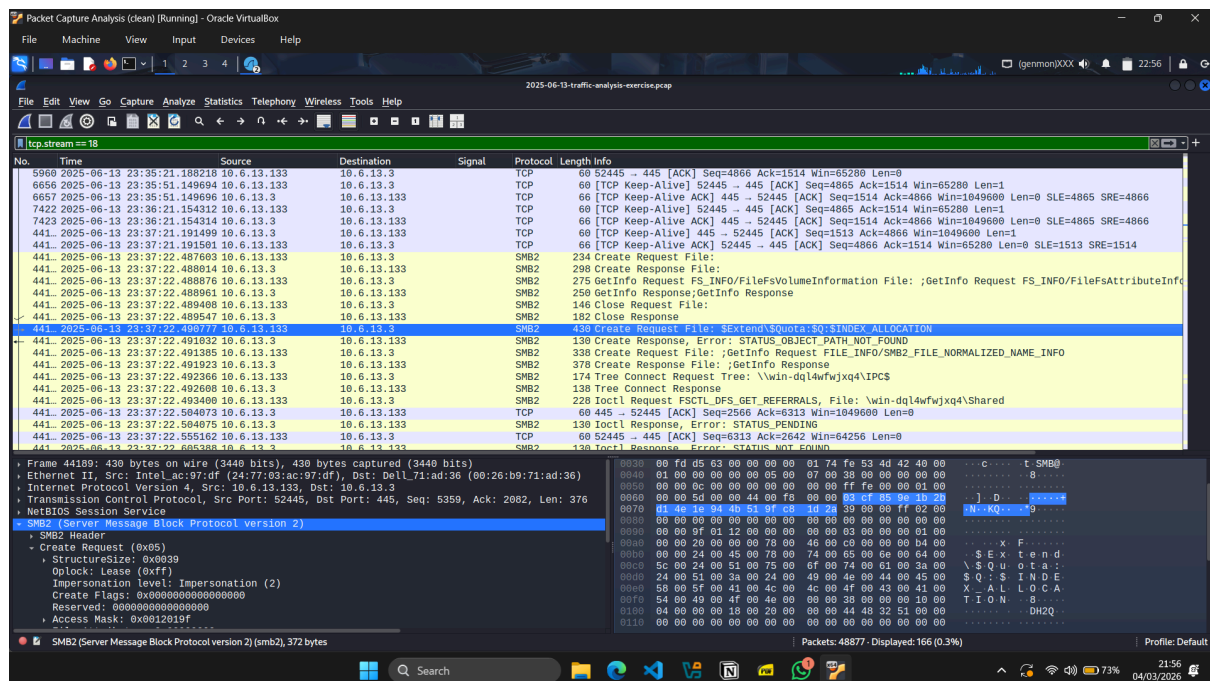
Protocol: UDP (17)
Header Checksum: 0xc620 [validation disabled]
[Header checksum status: Unverified]
2025-06-13-traffic-analysis-exercise.pcap

From the screenshot below, .133 connects to .3 for about 2041 seconds which when divided by 60 seconds per minute, we get about 34 minutes. The total packet is 166, which is considered to be relatively small for that duration. The destination port (port B) may also look suspicious since it is going to port 445, which is Server Message Block, therefore we will have to follow the stream to find out more about that specific stream.

Wireshark - Follow TCP Stream (tcp.stream eq 18) - 2025-06-13-traffic-analysis-exercise.pcap

Stream	Direction	Protocol	Length	Info
1	10.6.13.3 → 10.6.13.3	SMB	100	...
2	10.6.13.3 → 10.6.13.3	SMB	100	...
3	10.6.13.3 → 10.6.13.3	SMB	100	...
4	10.6.13.3 → 10.6.13.3	SMB	100	...
5	10.6.13.3 → 10.6.13.3	SMB	100	...
6	10.6.13.3 → 10.6.13.3	SMB	100	...
7	10.6.13.3 → 10.6.13.3	SMB	100	...
8	10.6.13.3 → 10.6.13.3	SMB	100	...
9	10.6.13.3 → 10.6.13.3	SMB	100	...
10	10.6.13.3 → 10.6.13.3	SMB	100	...
11	10.6.13.3 → 10.6.13.3	SMB	100	...
12	10.6.13.3 → 10.6.13.3	SMB	100	...
13	10.6.13.3 → 10.6.13.3	SMB	100	...
14	10.6.13.3 → 10.6.13.3	SMB	100	...
15	10.6.13.3 → 10.6.13.3	SMB	100	...
16	10.6.13.3 → 10.6.13.3	SMB	100	...
17	10.6.13.3 → 10.6.13.3	SMB	100	...
18	10.6.13.3 → 10.6.13.3	SMB	100	...
19	10.6.13.3 → 10.6.13.3	SMB	100	...
20	10.6.13.3 → 10.6.13.3	SMB	100	...
21	10.6.13.3 → 10.6.13.3	SMB	100	...
22	10.6.13.3 → 10.6.13.3	SMB	100	...
23	10.6.13.3 → 10.6.13.3	SMB	100	...
24	10.6.13.3 → 10.6.13.3	SMB	100	...
25	10.6.13.3 → 10.6.13.3	SMB	100	...
26	10.6.13.3 → 10.6.13.3	SMB	100	...
27	10.6.13.3 → 10.6.13.3	SMB	100	...
28	10.6.13.3 → 10.6.13.3	SMB	100	...
29	10.6.13.3 → 10.6.13.3	SMB	100	...
30	10.6.13.3 → 10.6.13.3	SMB	100	...
31	10.6.13.3 → 10.6.13.3	SMB	100	...
32	10.6.13.3 → 10.6.13.3	SMB	100	...
33	10.6.13.3 → 10.6.13.3	SMB	100	...
34	10.6.13.3 → 10.6.13.3	SMB	100	...
35	10.6.13.3 → 10.6.13.3	SMB	100	...
36	10.6.13.3 → 10.6.13.3	SMB	100	...
37	10.6.13.3 → 10.6.13.3	SMB	100	...
38	10.6.13.3 → 10.6.13.3	SMB	100	...
39	10.6.13.3 → 10.6.13.3	SMB	100	...
40	10.6.13.3 → 10.6.13.3	SMB	100	...
41	10.6.13.3 → 10.6.13.3	SMB	100	...
42	10.6.13.3 → 10.6.13.3	SMB	100	...
43	10.6.13.3 → 10.6.13.3	SMB	100	...
44	10.6.13.3 → 10.6.13.3	SMB	100	...
45	10.6.13.3 → 10.6.13.3	SMB	100	...
46	10.6.13.3 → 10.6.13.3	SMB	100	...
47	10.6.13.3 → 10.6.13.3	SMB	100	...
48	10.6.13.3 → 10.6.13.3	SMB	100	...
49	10.6.13.3 → 10.6.13.3	SMB	100	...
50	10.6.13.3 → 10.6.13.3	SMB	100	...
51	10.6.13.3 → 10.6.13.3	SMB	100	...
52	10.6.13.3 → 10.6.13.3	SMB	100	...
53	10.6.13.3 → 10.6.13.3	SMB	100	...
54	10.6.13.3 → 10.6.13.3	SMB	100	...
55	10.6.13.3 → 10.6.13.3	SMB	100	...
56	10.6.13.3 → 10.6.13.3	SMB	100	...
57	10.6.13.3 → 10.6.13.3	SMB	100	...
58	10.6.13.3 → 10.6.13.3	SMB	100	...
59	10.6.13.3 → 10.6.13.3	SMB	100	...
60	10.6.13.3 → 10.6.13.3	SMB	100	...
61	10.6.13.3 → 10.6.13.3	SMB	100	...
62	10.6.13.3 → 10.6.13.3	SMB	100	...
63	10.6.13.3 → 10.6.13.3	SMB	100	...
64	10.6.13.3 → 10.6.13.3	SMB	100	...
65	10.6.13.3 → 10.6.13.3	SMB	100	...
66	10.6.13.3 → 10.6.13.3	SMB	100	...
67	10.6.13.3 → 10.6.13.3	SMB	100	...
68	10.6.13.3 → 10.6.13.3	SMB	100	...
69	10.6.13.3 → 10.6.13.3	SMB	100	...
70	10.6.13.3 → 10.6.13.3	SMB	100	...
71	10.6.13.3 → 10.6.13.3	SMB	100	...
72	10.6.13.3 → 10.6.13.3	SMB	100	...
73	10.6.13.3 → 10.6.13.3	SMB	100	...
74	10.6.13.3 → 10.6.13.3	SMB	100	...
75	10.6.13.3 → 10.6.13.3	SMB	100	...
76	10.6.13.3 → 10.6.13.3	SMB	100	...
77	10.6.13.3 → 10.6.13.3	SMB	100	...
78	10.6.13.3 → 10.6.13.3	SMB	100	...
79	10.6.13.3 → 10.6.13.3	SMB	100	...
80	10.6.13.3 → 10.6.13.3	SMB	100	...
81	10.6.13.3 → 10.6.13.3	SMB	100	...
82	10.6.13.3 → 10.6.13.3	SMB	100	...
83	10.6.13.3 → 10.6.13.3	SMB	100	...
84	10.6.13.3 → 10.6.13.3	SMB	100	...
85	10.6.13.3 → 10.6.13.3	SMB	100	...
86	10.6.13.3 → 10.6.13.3	SMB	100	...
87	10.6.13.3 → 10.6.13.3	SMB	100	...
88	10.6.13.3 → 10.6.13.3	SMB	100	...
89	10.6.13.3 → 10.6.13.3	SMB	100	...
90	10.6.13.3 → 10.6.13.3	SMB	100	...
91	10.6.13.3 → 10.6.13.3	SMB	100	...
92	10.6.13.3 → 10.6.13.3	SMB	100	...
93	10.6.13.3 → 10.6.13.3	SMB	100	...
94	10.6.13.3 → 10.6.13.3	SMB	100	...
95	10.6.13.3 → 10.6.13.3	SMB	100	...
96	10.6.13.3 → 10.6.13.3	SMB	100	...
97	10.6.13.3 → 10.6.13.3	SMB	100	...
98	10.6.13.3 → 10.6.13.3	SMB	100	...
99	10.6.13.3 → 10.6.13.3	SMB	100	...
100	10.6.13.3 → 10.6.13.3	SMB	100	...
101	10.6.13.3 → 10.6.13.3	SMB	100	...
102	10.6.13.3 → 10.6.13.3	SMB	100	...
103	10.6.13.3 → 10.6.13.3	SMB	100	...
104	10.6.13.3 → 10.6.13.3	SMB	100	...
105	10.6.13.3 → 10.6.13.3	SMB	100	...
106	10.6.13.3 → 10.6.13.3	SMB	100	...
107	10.6.13.3 → 10.6.13.3	SMB	100	...
108	10.6.13.3 → 10.6.13.3	SMB	100	...
109	10.6.13.3 → 10.6.13.3	SMB	100	...
110	10.6.13.3 → 10.6.13.3	SMB	100	...
111	10.6.13.3 → 10.6.13.3	SMB	100	...
112	10.6.13.3 → 10.6.13.3	SMB	100	...
113	10.6.13.3 → 10.6.13.3	SMB	100	...
114	10.6.13.3 → 10.6.13.3	SMB	100	...
115	10.6.13.3 → 10.6.13.3	SMB	100	...
116	10.6.13.3 → 10.6.13.3	SMB	100	...
117	10.6.13.3 → 10.6.13.3	SMB	100	...
118	10.6.13.3 → 10.6.13.3	SMB	100	...
119	10.6.13.3 → 10.6.13.3	SMB	100	...
120	10.6.13.3 → 10.6.13.3	SMB	100	...
121	10.6.13.3 → 10.6.13.3	SMB	100	...
122	10.6.13.3 → 10.6.13.3	SMB	100	...
123	10.6.13.3 → 10.6.13.3	SMB	100	...
124	10.6.13.3 → 10.6.13.3	SMB	100	...
125	10.6.13.3 → 10.6.13.3	SMB	100	...
126	10.6.13.3 → 10.6.13.3	SMB	100	...
127	10.6.13.3 → 10.6.13.3	SMB	100	...
128	10.6.13.3 → 10.6.13.3	SMB	100	...
129	10.6.13.3 → 10.6.13.3	SMB	100	...</

After following the TCP stream, it turns out to be encrypted. But I tried looking at it more thoroughly, by setting up a filter in Wireshark, which is "tcp.stream == 18", since it is what is said when we follow the TCP stream.



When I saw the packets I immediately looked for familiar names, such as:

- \srvsvc or \lsarpc - standard windows pipes
- \msagent_x - default name for cobalt strike beacons
- \status_x - often used by Metasploit framework
- \randomstring - mostly used to avoid detection for C2.

The first thing I noticed is that the Create Request File, Get Info, Tree Connect which signals that there is an initiating SMB connection.

But what I found is the \$Extend\$Quota directory which is a hidden directory, an important NTFS system metadata file located in the root of the drive, that stores information about users' accounts. But then it responded with a path not found error status message.

Then it also done a Get Info request to:

- FS_INFO/FileFsVolume Information
- FileNormalizedNamesInformation

Rafael Angelo Christianto

It gathers information about file attributes, file volumes, and the type of file system. So from the information we got previously, this is not a beaconing behavior. Neither it is a payload, exploit, or file upload. This shows an SMB enumeration activity. That is for now.

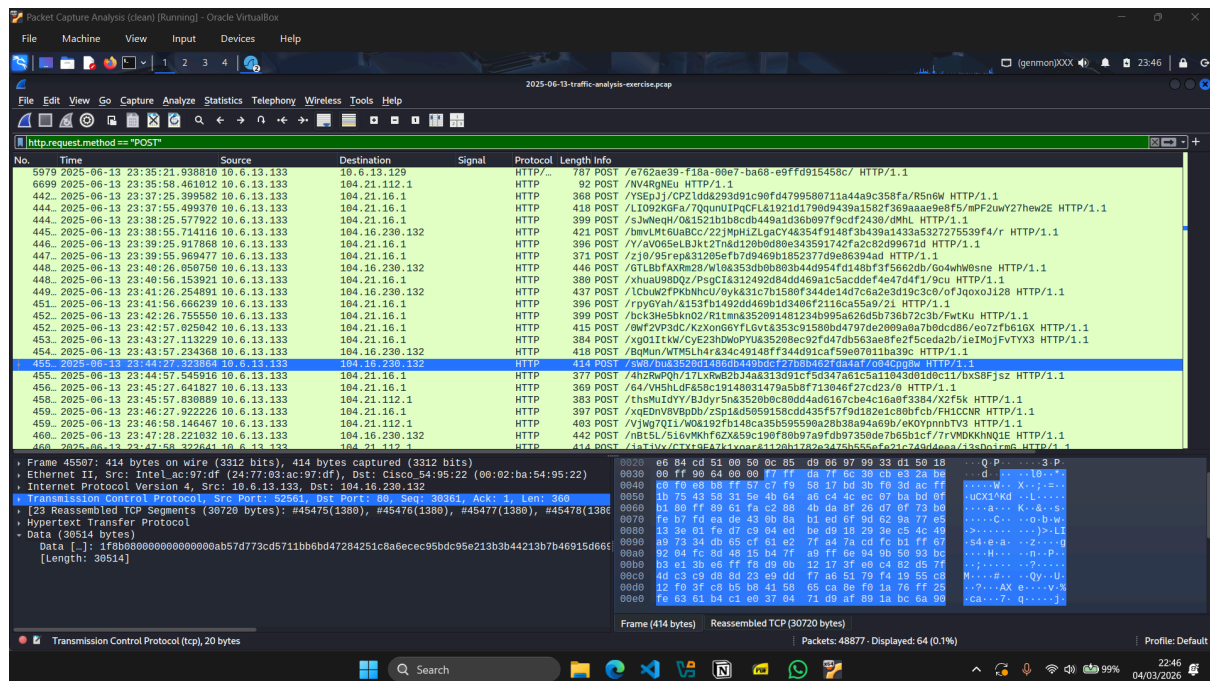
But I still want to check if there are any Beaconing or C2 activity, and then I want to filter for POST requests since C2 behavior will always be POST requests to the same IP address regularly, at an odd time interval, and similar packet sizes. And here we can see data is posted to 104.21.16.1 and 104.16.230.132. On the left side columns, on the Column of "Time", we can see the time interval is usually 30 seconds, which would indicate a C2 action.

The screenshot displays the Wireshark interface with a packet capture of traffic on the interface '2025-06-13-traffic-analysis-exercise.pcap'. The packet list on the left shows a series of HTTP POST requests to various IP addresses, including 10.6.13.129, 104.21.112.1, 104.21.16.1, and 104.16.230.132. The time intervals between these requests are consistently 30 seconds. The packet details pane on the right shows the structure of a selected packet, including the Ethernet II header, Internet Protocol Version 4 header, and Transmission Control Protocol (TCP) header. The TCP header indicates a sequence number of 30361 and an acknowledgment number of 1. The packet length is 414 bytes.

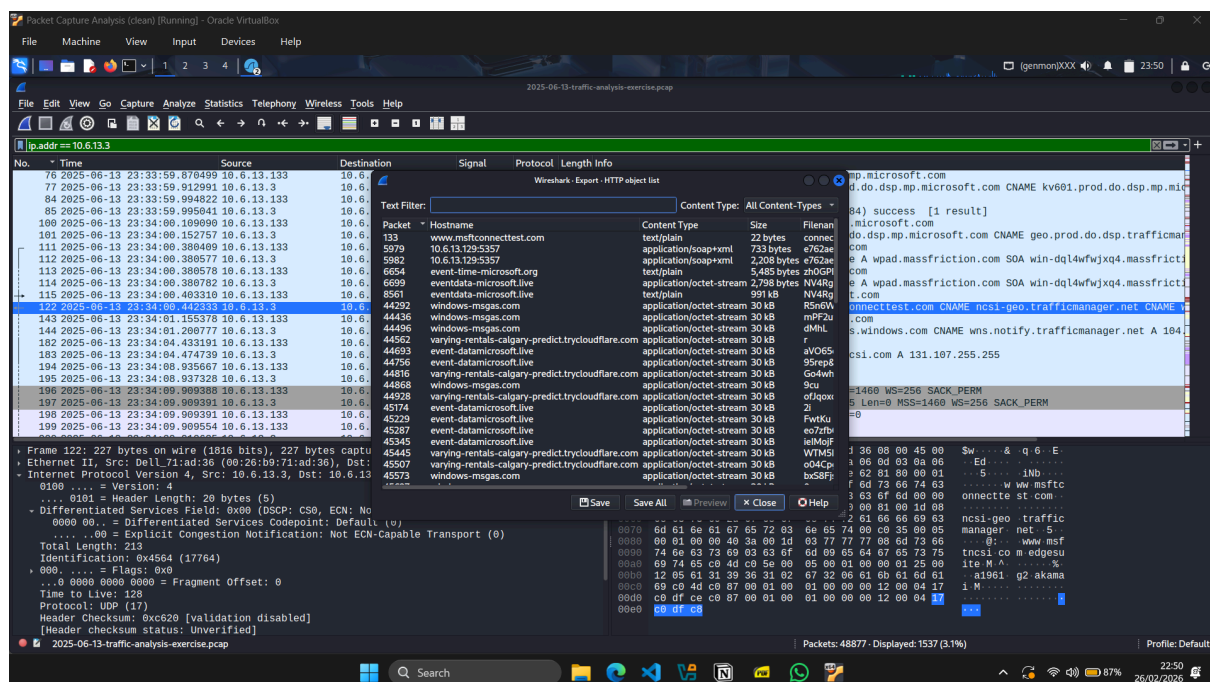
No.	Time	Source	Destination	Signal	Protocol	Length	Info
5979	2025-06-13 23:35:21.938018	10.6.13.133	10.6.13.129	HTTP	POST	787	/f/62a39-f18a-80e7-ba68-e9ff0915458c/ HTTP/1.1
6099	2025-06-13 23:35:58.461812	10.6.13.133	104.21.112.1	HTTP	POST	92	/N/4RqNEU HTTP/1.1
442	2025-06-13 23:37:25.399582	10.6.13.133	104.21.16.1	HTTP	POST	368	/YSEpJ/CPZ1dd&29d91c90fd4799580711a44a9c358fa/R5n6w HTTP/1.1
444	2025-06-13 23:37:55.499370	10.6.13.133	104.21.16.1	HTTP	POST	418	/L1092K6Fa/7qquuIPqCFL1921d1790d9439a1582f369aaae9e8f5/mPF2uwy72hew2E HTTP/1.1
444	2025-06-13 23:38:25.577922	10.6.13.133	104.21.16.1	HTTP	POST	399	/s3wWqW/0A1521b1b8cb449a4f4b0f7f5662db/Gc4whW8ne HTTP/1.1
445	2025-06-13 23:38:55.714116	10.6.13.133	104.16.230.132	HTTP	POST	421	/bmvLMtEuaB8C/22JMpHIZLgaCY46354f9148f3b439a1433a532725539f4/r HTTP/1.1
446	2025-06-13 23:39:25.917868	10.6.13.133	104.21.16.1	HTTP	POST	396	/Y/av05eL8Jkt2Tn&d128b0d80e343591742fa2c82d99671d HTTP/1.1
447	2025-06-13 23:39:55.968477	10.6.13.133	104.21.16.1	HTTP	POST	371	/z10/95rep431295efb709469b1852377d9e86394ad HTTP/1.1
448	2025-06-13 23:40:26.069750	10.6.13.133	104.16.230.132	HTTP	POST	446	/LbLfaXvR28/vl0a353db0b083b4440954f414b0f7f5662db/Gc4whW8ne HTTP/1.1
448	2025-06-13 23:40:56.153921	10.6.13.133	104.21.16.1	HTTP	POST	380	/xhuau98DQz/PspC1&32492d84dd469a1c5acdddef4e47d4f1/9cu HTTP/1.1
449	2025-06-13 23:41:26.254891	10.6.13.133	104.16.230.132	HTTP	POST	437	/LcbuW2FPkbNhcU/8yk&31c7b1580f344de14d7c6a2e3d19c3c8/ofJqoxoJ128 HTTP/1.1
451	2025-06-13 23:41:56.666239	10.6.13.133	104.21.16.1	HTTP	POST	396	/rpyGyah/41537b1492dd469b1d3486f2116ca55a9/21 HTTP/1.1
452	2025-06-13 23:42:26.755580	10.6.13.133	104.21.16.1	HTTP	POST	399	/bck3whe3bn02/R1tma&352091481234b995a626d5b780b72c3b/FwKku HTTP/1.1
452	2025-06-13 23:42:57.862542	10.6.13.133	104.21.16.1	HTTP	POST	415	/9wF2VP3dC/Kzx0n6GyflGvL&353c91588bd4797de2809a8a7b0dcdb6/eo7zfb616X HTTP/1.1
453	2025-06-13 23:43:27.113229	10.6.13.133	104.21.16.1	HTTP	POST	384	/xg01ITkW/CyE23h0W0PYu&35208ec92fd47d0b563ae8fe2fceda2b/1eIMoJFvTYX HTTP/1.1
454	2025-06-13 23:43:57.234368	10.6.13.133	104.16.230.132	HTTP	POST	418	/BqMun/WTMSLH4r&34c49148ff344d91caf59e07011ba39c HTTP/1.1
455	2025-06-13 23:44:27.622884	10.6.13.133	104.16.230.132	HTTP	POST	414	/xv0vYmS39C04f&30b412000f2f3508f94641f703400p HTTP/1.1
455	2025-06-13 23:44:57.545916	10.6.13.133	104.21.16.1	HTTP	POST	377	/4h2rWpQH/17LxRwB2b34a&313d91cf5d347a61c5a11943d01d8c17/bvS8Fjsz HTTP/1.1
456	2025-06-13 23:45:27.641827	10.6.13.133	104.21.16.1	HTTP	POST	369	/64/VH5HlDf&58c19148031479a5b8f713046f27cd23/0 HTTP/1.1
458	2025-06-13 23:45:57.838889	10.6.13.133	104.21.112.1	HTTP	POST	383	/thsMuIDvY/B3dyr5n&3520b0c80dd4ad6167che4c16a6f3384/X2f5k HTTP/1.1
459	2025-06-13 23:46:27.922226	10.6.13.133	104.21.16.1	HTTP	POST	397	/xqEDvrvBv0b/25p1a5959158cd0435f57f9d182e1c808fcb/PHICCR HTTP/1.1
459	2025-06-13 23:46:58.146467	10.6.13.133	104.21.112.1	HTTP	POST	483	/VJwQ11/W0k192fb148ca35b595598a28b38a94a69b/ekOYpnnbTV3 HTTP/1.1
460	2025-06-13 23:47:28.221032	10.6.13.133	104.16.230.132	HTTP	POST	442	/nbt5L/51vMKHf6Z&59c190f80b97a9fd897350de7b65b1cf/7rVMDKKNQ1E HTTP/1.1
460	2025-06-13 23:47:58.322641	10.6.13.133	104.21.112.1	HTTP	POST	414	/4Pt4Uv/QT71dE3V14v0r2412b17a9a342b556e7a21c2A6dA9a/13b0tzm HTTP/1.1

Frame 4597: 414 bytes on wire (3312 bits), 414 bytes captured (3312 bits) on interface 2025-06-13-traffic-analysis-exercise.pcap
Ethernet II, Src: Intel_ac:97:df:24 (24:77:09:ac:97:df), Dst: Cisco:54:95:22 (08:02:ba:54:95:22)
Internet Protocol Version 4, Src: 10.6.13.133, Dst: 104.16.230.132
Transmission Control Protocol, Src Port: 52561, Dst Port: 80, Seq: 30361, Ack: 1, Len: 360
Source Port: 52561
Destination Port: 80
[Stream index: 135]
[Stream Packet Number: 36]
[Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 360]
Sequence Number: 30361 (relative sequence number)
Sequence Number (raw): 210698438
[Next Sequence Number: 30721 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
[ACK Flag Set]

Then I tried looking again at the packet sizes and they were similarly uniform in sizes, which usually are in the 30500s bytes. This is the confirmation already that this is a beaconing or C2 behavior.



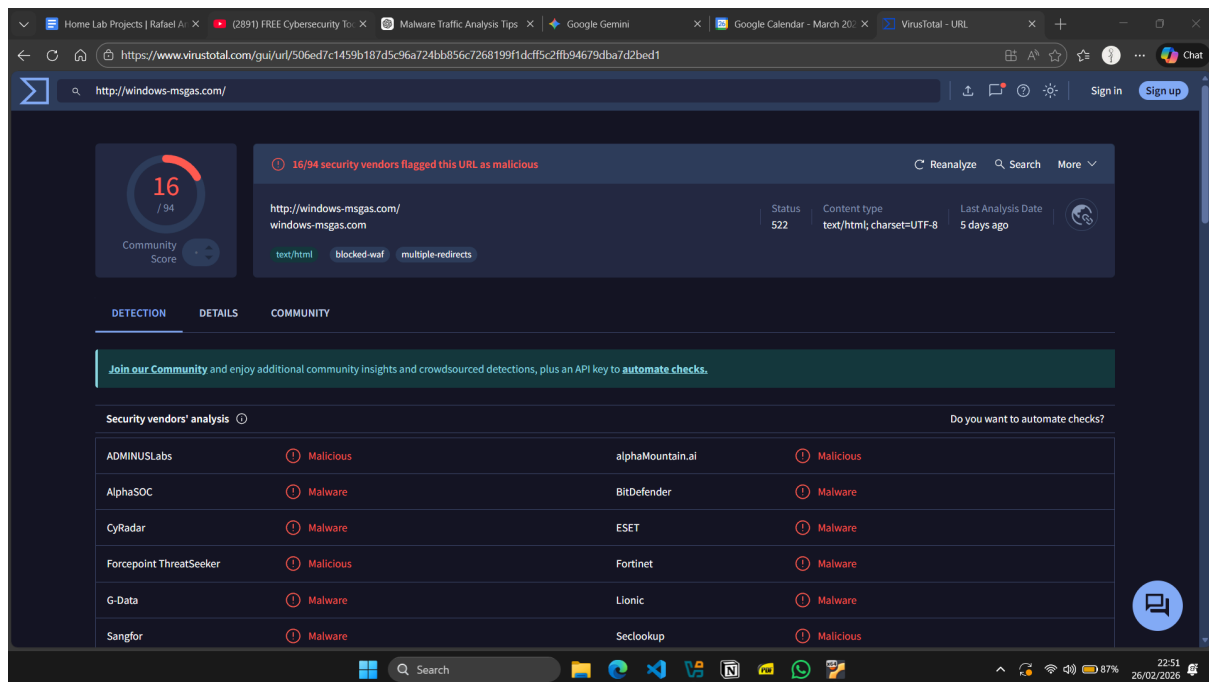
Below, I went ahead and saw the Export Objects feature from Wireshark, but I will not be saving the file as it could cause harm and damage, even though I am already working on a VM. This menu is accessible from File > Export Objects > HTTP. Therefore these files/links are associated with HTTP throughout this packet capture file. We can see several links that may look malicious that we will explore soon.



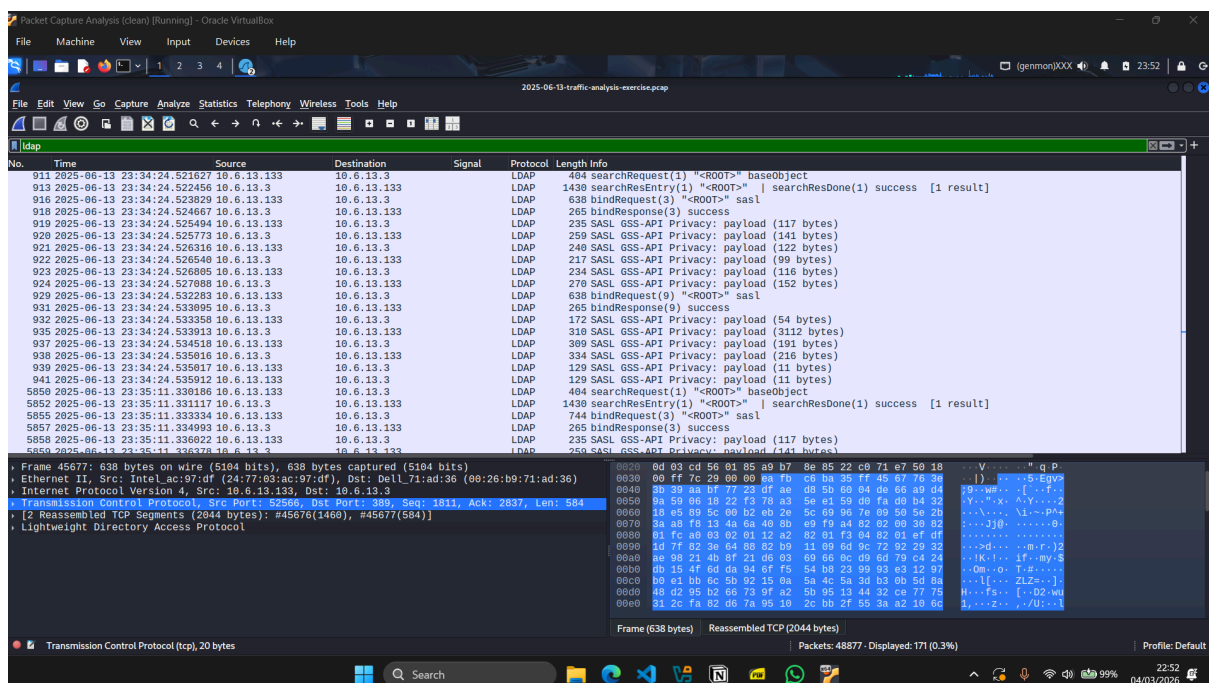
First is that of course, there is the event-time-microsoft.org that we scanned already previously. There is also microsoft-msgas.com, which does not sound or

Rafael Angelo Christianto

look like a legit microsoft domain, therefore I will just scan it with Virus Total and yes it is marked as malicious.



Now I want to check for Active Directory enumeration by filtering ldap and it is happening between 10.6.13.133 and 10.6.13.3.



As we can see from the screenshot above, the client is querying the root of the directory tree, which is commonly used for the retrieval of the domain name. And the client is authenticating through LDAP through SASL (Kerberos).

But more to the bottom packets in the screenshot above, we see that the connection is encrypted with GSS-API which is a Kerberos session protection. But since I can see only a few LDAP request packets and there is only a total of 171 packets for LDAP, I have concluded that there is no DNS enumeration involved here.

Conclusion

Infected IP: 10.6.13.133

MAC: 24:77:03:ac:97:df

Hostname: DESKTOP-5AVE44C

User: rgaines

C2 Domains:

- event-time-microsoft.org
- microsoft-msgas.com

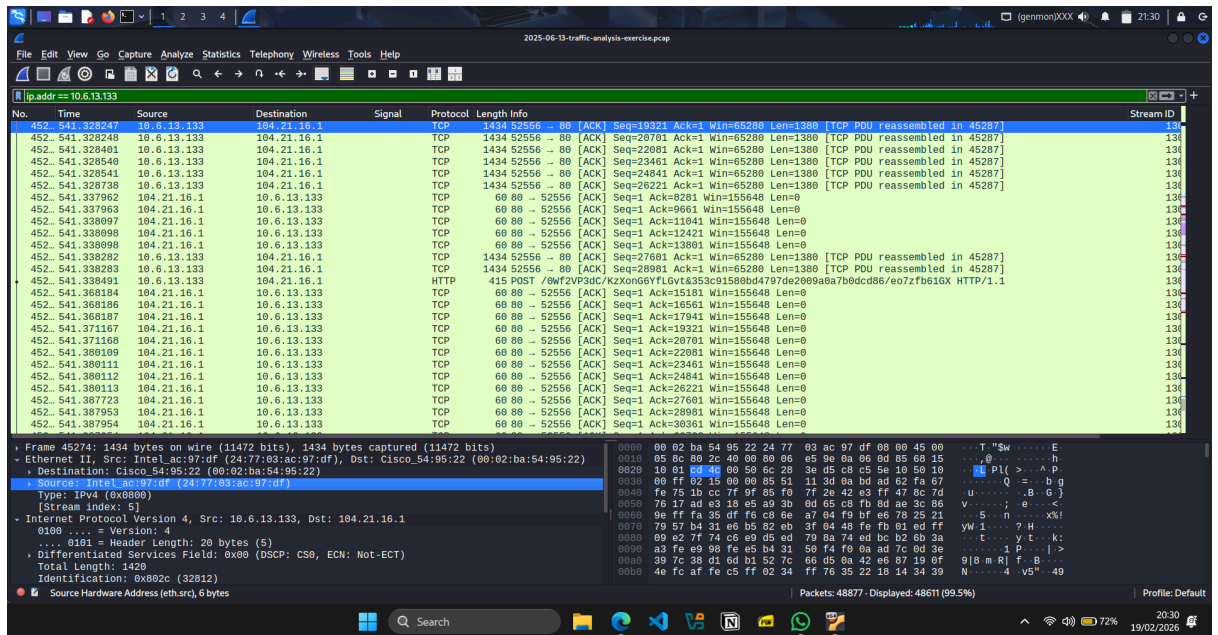
C2 IPs:

- 104.21.16.1
- 104.16.230.132

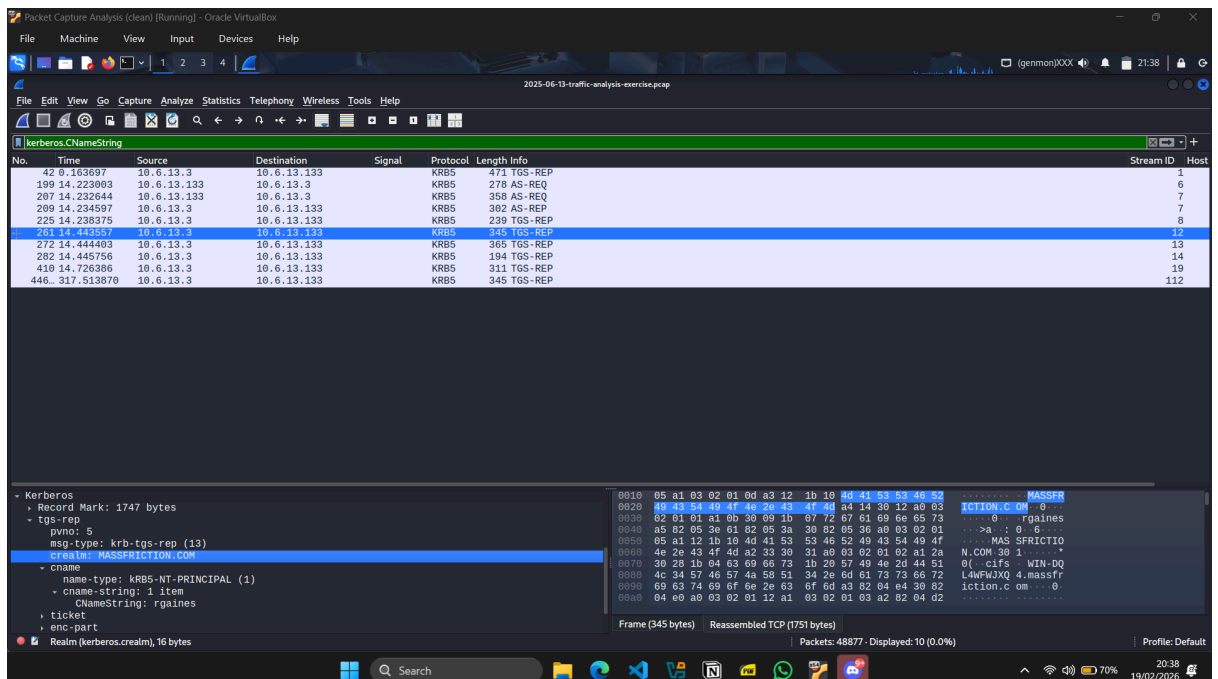
The Questions

The questions below are provided by malware-traffic-analysis.net specifically to this packet capture file. There are 4 and here are the answers:

- What is the IP address of the infected Windows client?
 - Thorough the packet capture file, we can notice that 10.6.13.133 is the most targeted IP address
- What is the mac address of the infected Windows client?



- I put a filter to the IP of the infected client, which is 'ip.addr == 10.6.13.133'. And in the bottom left under the Ethernet dropdown which would indicate Layer 2, the Data Link layer. The blocked blue row indicates the MAC address of the infected client which is, 24:77:03:ac:97:df.
- What is the host name of the infected Windows client?
 - Like we have found before, by massfriction.com, the host name is DESKTOP-5AVE44C.
- What is the user account name from the infected Windows client?



- I found the username using Kerberos, which is one of the protocols to authenticate. And the syntax is 'Kerberos.CNameString'. In the left bottom, we can see the username which is rgaines.